

UNIVERSITÉ MARIE ET LOUIS PASTEUR

Master 2 Systèmes Communicants pour l'Internet des Objets
UE 7 Mobility in Smart Cities

Make Change Practical Work Report

Greedy, Recursive Enumeration, Dynamic Programming and Exact Search

Item	Information
Students	Ahmed Al-Muharaq and Owais Khan
Instructor	Prof. Philippe Canalda
Academic year	2025–2026
Submission	7 October 2025
Implementation	C++17

Montbéliard, France

Abstract

In this practical work, we study the Make Change problem and implement the requested methods in C++17. We first apply greedy selection to ordered and unordered denomination lists. We then use recursion to generate every valid solution, introduce the coin-count fitness objective, print only solutions that improve the incumbent, and add safe branch-and-bound cuts. We also use bottom-up dynamic programming as an independent verification of the minimum. For the official amount EUR 12.35, complete recursion finds 266,724 valid solutions after exploring 12,607,231 states. The exact search with cuts proves the six-coin optimum after only 18 states. Measured curves confirm linear greedy scanning and rapid combinatorial growth for complete recursion.

Contents

- 1 Scope and requested work
- 2 Team and contributions
- 3 Problem model and design
- 4 Greedy Make Change
- 5 Recursive generation of all valid solutions
- 6 Fitness objective, improving solutions and cuts
- 7 Dynamic-programming verification
- 8 C++ implementation
- 9 Tests and measured results
- 10 Complexity curves and evaluation
- 11 Conclusion
- References

1 Scope and requested work

We restrict this report to the Make Change practical work. The separate mini-project is not included. We translated the board instructions into the following deliverables.

- We implement greedy Make Change with an ordered list and then with an unordered list, without limiting how many times a denomination may be used.
- We provide a commented C++17 program, executions and a curve that demonstrates the linear greedy scan.
- We implement a recursive method that generates all valid solutions and show its combinatorial/exponential growth.
- We integrate a fitness objective that minimizes the number of coins, print only improving solutions, and cut branches that cannot beat the incumbent.
- We test, compare and explain every method using the official case and counterexamples.

2 Team and contributions

We, Ahmed Al-Muharaq and Owais Khan, carried out this TP jointly. We discussed the model, implemented and reviewed the C++ code, selected counterexamples, executed the benchmarks, generated the curves and checked the conclusions. Our Git history should preserve the final individual commit record used for the Moodle submission.

Contribution	Responsibility
Problem model and algorithm design	Joint work
C++ implementation and comments	Joint implementation and review
Tests, benchmarks and curves	Joint execution and verification
Analysis and report	Joint writing and review

3 Problem model and design

3.1 Official instance

We must return $M = \text{EUR } 12.35$ with $L = [5, 2, 1, 0.50, 0.20, 0.10, 0.05]$. We represent a solution by a vector S . Each $S[i]$ is the number of occurrences of $L[i]$, and every count is unbounded except by the remaining amount. A valid solution satisfies the following equation:

$$\sum S[i] \times L[i] = M, \text{ with } S[i] \geq 0 \text{ and integer}$$

3.2 Exact monetary representation

We store euros as integer cents: the coin list becomes $[500, 200, 100, 50, 20, 10, 5]$ and the target becomes 1235. This avoids binary floating-point rounding errors during subtraction, equality tests and divisibility checks. We validate positive denominations, remove duplicates and sort only when the selected method requires normalization.

3.3 Solution quality

Our fitness value is the total number of coins $f(S) = \sum S[i]$. A smaller value is better. We separate three questions: whether a solution exists, whether all solutions were generated, and whether the minimum number of coins was proved. This distinction prevents us from treating the first valid solution as automatically optimal.

4 Greedy Make Change

4.1 Rule and loop invariant

We scan the denomination list from left to right. For each coin, we take the maximum possible count, update the remainder, and continue. After processing position i , our count vector records all greedy decisions from 0 to i and the remainder is smaller than $L[i]$. This invariant proves that the program follows the greedy rule, but it does not prove global optimality.

```
for each coin c in the current list:
    count = remaining / c
    remaining = remaining % c
    store count
```

4.2 Ordered official list

Coin	Count	Selected value	Remainder
EUR 5.00	2	EUR 10.00	EUR 2.35
EUR 2.00	1	EUR 2.00	EUR 0.35
EUR 1.00	0	EUR 0.00	EUR 0.35
EUR 0.50	0	EUR 0.00	EUR 0.35
EUR 0.20	1	EUR 0.20	EUR 0.15
EUR 0.10	1	EUR 0.10	EUR 0.05
EUR 0.05	1	EUR 0.05	EUR 0.00

We obtain $S = [2, 1, 0, 0, 1, 1, 1]$, which returns EUR 12.35 with six coins.

4.3 Unordered list

We deliberately use $[0.20, 5, 0.50, 1, 0.05, 2, 0.10]$. If we apply the greedy rule directly to this order, it returns the correct amount with 64 coins. If we first sort the same values in decreasing order, it returns the six-coin solution. The direct scan is $O(n)$; normalization by comparison sorting makes the complete operation $O(n \log n)$. The experiment shows that list order matters, but sorting alone still cannot guarantee that greedy is optimal for every coin system.

4.4 Counterexamples

Coins	Amount	Greedy result	Exact result
[4, 3, 1]	6	4+1+1: 3 coins	3+3: 2 coins
[5, 2, 1.5]	9.5	Failure with 0.5 left	5+1.5+1.5+1.5: 4 coins

Therefore, we use greedy as a fast heuristic and as an initial upper bound, not as a general proof of optimality.

5 Recursive generation of all valid solutions

5.1 Recursive representation

At recursion level i , we choose the number of occurrences of $L[i]$. The possible count ranges from $\text{floor}(\text{remaining}/L[i])$ down to zero. We then recurse to $i+1$ with the reduced remainder. The partial vector is extended before the call and restored afterward; this is backtracking.

```
enumerate(i, remaining):
    if remaining == 0: record the completed vector
    else if i == number of coins: stop this branch
    else:
        for count from remaining / coin[i] down to 0:
            choose count
            enumerate(i + 1, remaining - count * coin[i])
            undo the choice
```

5.2 Stop conditions and correctness

- Success: when the remainder is zero, we complete the unused positions with zero and count one valid solution.
- Failure: when every denomination was processed and the remainder is still positive, we reject the branch.
- Completeness: at every level, we try every legal count, so every valid count vector has one root-to-leaf path.
- Uniqueness: two different paths differ in at least one count position, so we do not generate duplicates.

5.3 Cost of complete generation

For the official instance, our recursion visits 12,607,231 states and finds 266,724 valid solutions. If every solution were printed, console and file output would add a large I/O cost on top of the search itself. The number of count combinations is bounded by a product of the choices available at each level, so the worst case is exponential in the number of denominations. The recursion depth itself is only $O(n)$, but storing every solution requires output-sensitive memory.

6 Fitness objective, improving solutions and cuts

6.1 Printing only improvements

We maintain the best coin count found so far. A valid solution is printed and stored only if its fitness is strictly smaller than the incumbent. This changes the output from every valid solution to a short sequence of improving incumbents. It saves the time required to format and print non-improving solutions, but by itself it does not reduce the number of recursive states.

6.2 Safe branch-and-bound cut

We initialize the incumbent with the greedy solution whenever greedy succeeds. At index i , all remaining coins are no larger than $L[i]$. Even under the optimistic assumption that every future coin has value $L[i]$, we still need at least $\text{ceil}(\text{remaining}/L[i])$ additional coins. We cut when:

$$\text{used} + \text{ceil}(\text{remaining} / L[i]) \geq \text{bestCount}$$

This lower bound can underestimate the true number of required coins, but it never overestimates it. Consequently, any cut branch cannot contain a solution strictly better than the incumbent. We also stop when $\text{used} \geq \text{bestCount}$ and use the greatest common divisor to reject amounts that cannot be composed from the remaining denominations. These cuts preserve exactness.

7 Dynamic-programming verification

We also implement bottom-up dynamic programming to verify the exact minimum independently. For every amount x from 1 to M , we compute the minimum coins needed for x from previously solved amounts $x-c$. We store the last selected coin to reconstruct a solution.

```
dp[0] = 0
for x from 1 to amount:
    for each coin c with c <= x:
        dp[x] = min(dp[x], dp[x - c] + 1)
```

With n denominations and an amount M expressed in cents, this verification costs $O(nM)$ time and $O(M)$ memory. It is pseudo-polynomial because its cost depends on the numeric amount. It finds one optimal solution; it does not enumerate all valid solutions. This distinction is why we describe the course recursion as complete recursive enumeration and use dynamic programming as a separate verification method. Both return the same six-coin optimum for EUR 12.35.

8 C++ implementation

8.1 Program structure

C++ function	Responsibility
normalizeCoins	Validate, deduplicate and sort denominations
greedyAssumingCurrentOrder	Apply greedy without hiding the effect of the input order
greedyWithNormalization	Sort an unordered list before greedy selection
enumerateRecursive	Generate every valid solution recursively
bestSolutionWithCuts	Minimize coin count and apply safe recursive cuts
dynamicProgrammingMinimum	Verify the minimum with bottom-up DP
writeGreedyBenchmark / writeRecursiveBenchmark	Generate reproducible CSV data for the curves

8.2 Compilation and execution

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic make_change.cpp -o make_change
./make_change
./make_change --benchmark
python3 plot_make_change_curves.py
```

We compile with warnings enabled and optimization level $O2$. The normal execution prints the official case and counterexamples. The benchmark mode creates the two CSV files consumed by the plotting script. All code comments use our voice and explain the reason for each important choice.

8.3 Repository

We provide `make_change.cpp`, the plotting script, benchmark CSV files and generated figures with this report. We will add the final GitHub repository URL to the Moodle submission before upload.

9 Tests and measured results

9.1 Functional tests

Test	Input	Observed result	What we verify
Official ordered	coins [500..5], M=1235	Greedy and exact: 6 coins	Correct official answer
Official unordered raw	[20,500,50,100,5,200,10]	64 coins	Order affects greedy
Official unordered sorted	same list, normalized	6 coins	Normalization works
Nonoptimal greedy	[400,300,100], M=600	Greedy 3; exact 2	First/greedy is not always best
Greedy failure	[500,200,150], M=950	Greedy fails; exact uses 4	Exact search finds missed solution

9.2 Official search measurements

Method	Explored states	Valid/output solutions	Result
Greedy scan	7 denomination steps	1 candidate	6 coins
Complete recursion	12,607,231	266,724 valid	All solutions
Exact recursion with cuts	18	1 incumbent	6 coins, proven optimal
Bottom-up DP	7×1235 transitions at most	1 reconstructed optimum	6 coins

We prioritize state counts because they are reproducible across machines, while wall-clock time changes with the processor, compiler and system load. The large reduction from complete recursion to exact recursion with cuts is the main experimental result.

10 Complexity curves and evaluation

10.1 Greedy linearity

We vary the number of denominations from 10 to 5,000 and count the ordered greedy loop iterations. Every denomination is inspected exactly once, so the measured curve is $y=n$. This confirms $O(n)$ scanning time. If the input must first be sorted, the complete normalized method has $O(n \log n)$ time.

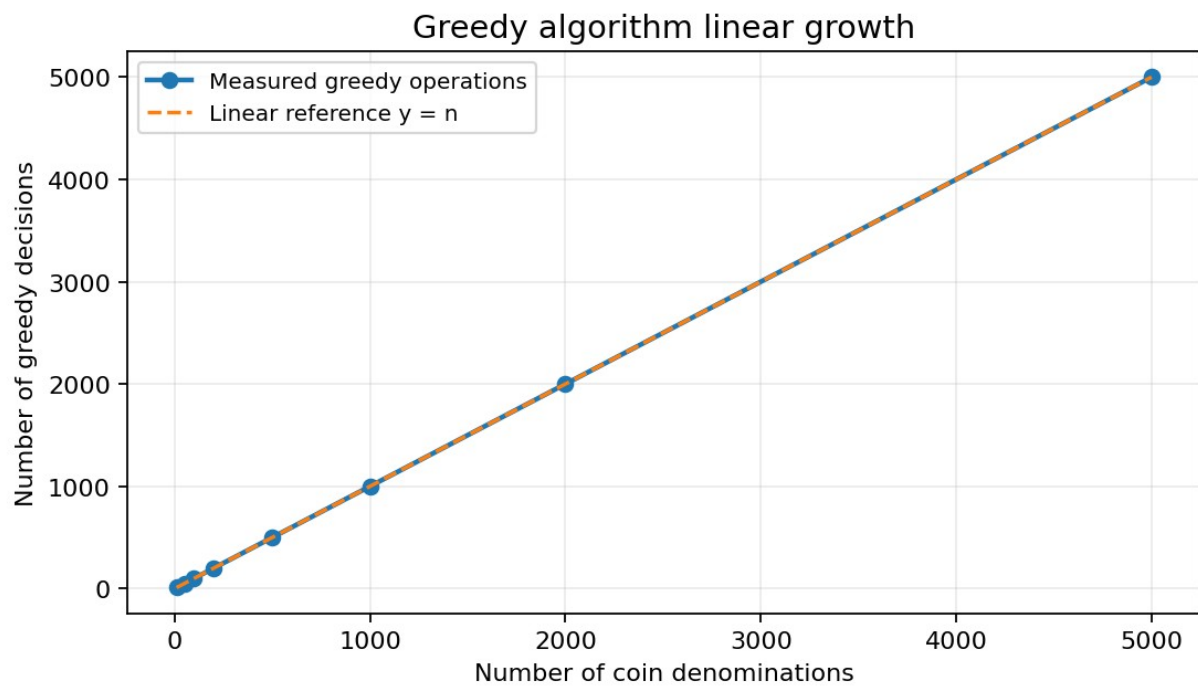


Figure 1 — Measured ordered greedy operations grow linearly with the number of denominations.

10.2 Growth of complete recursion

For the growth benchmark, we use n denominations $[n, n-1, \dots, 1]$ and target $3n$. We plot the number of recursive states and valid solutions on a logarithmic vertical axis. The state count rises from 21 for two denominations to 302,664 for fourteen denominations. The rapidly rising curve is consistent with the exponential worst-case bound of full enumeration.

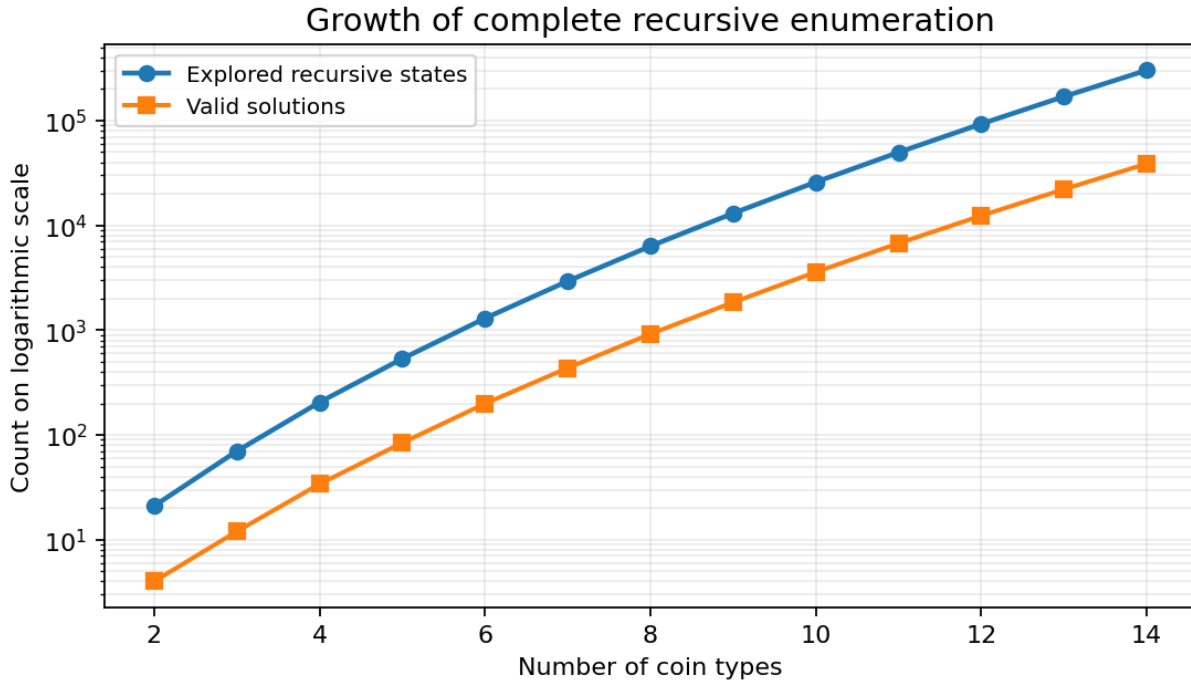


Figure 2 — Complete recursive enumeration grows rapidly; the vertical axis is logarithmic.

10.3 Effect of the cut

We run the same benchmark with the incumbent and lower-bound cut enabled. In these deliberately structured instances, greedy uses three maximum-value coins and the lower bound also proves that fewer than three coins are impossible. The root is therefore safely pruned after one state. We report this transparently: it is a best-case demonstration of a strong bound, not a claim that every input is solved in constant time. The worst case remains exponential.

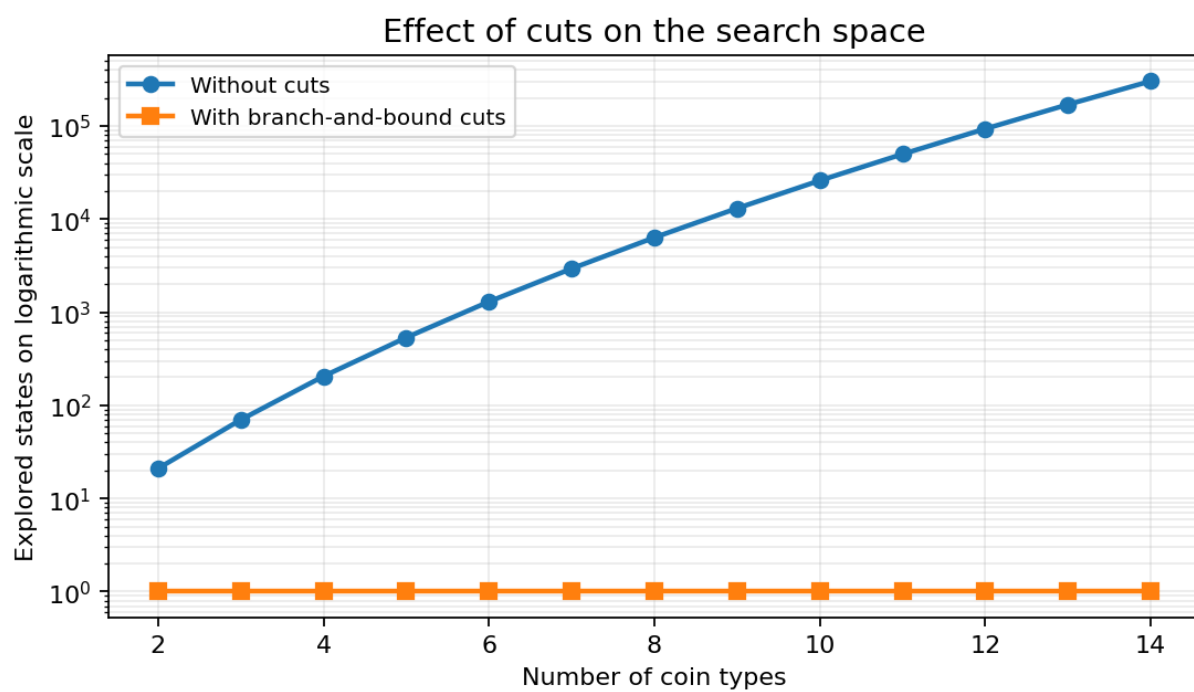


Figure 3 — Explored states with and without the exact branch-and-bound cut.

10.4 Comparative evaluation

Method	Guarantee	Main strength	Main limitation
Ordered greedy	No general optimum	$O(n)$, simple and fast	May fail or be nonoptimal
Complete recursion	All valid solutions	Complete reference result	Exponential growth and large output
Improving-only output	Same search guarantee	Avoids unnecessary printing	Does not alone reduce search states
Recursion with cuts	Optimal	Can prune most branches	Worst case is still exponential
Bottom-up DP	Optimal	Predictable $O(nM)$ verification	Pseudo-polynomial; does not list all solutions

Our results answer the main course questions. The first or greedy solution is sometimes the best, as in the official instance, but not always. Printing only improvements reduces output time. A cut is valid only when a lower bound proves that the branch cannot improve the incumbent. When that proof is unavailable, we continue exploring so that optimality is preserved.

11 Conclusion

We implemented and evaluated the complete Make Change TP in C++17. We showed that ordered greedy selection is linear and solves the official case with six coins, while an unordered scan can be much worse. We then generated all 266,724 valid solutions recursively and observed the expected combinatorial growth. By integrating the coin-count fitness, printing only improving incumbents and applying admissible cuts, we retained an exact optimum while reducing the official search from 12,607,231 states to 18. Finally, bottom-up dynamic programming independently confirmed the same optimum. The experiments demonstrate the trade-off between speed, completeness and proof of optimality.

References

- [1] P. Canalda, MSC Dynamic Transportation Exact Solving, UE 7 Mobility in Smart Cities, course slides, 23 September 2025.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.